
COMANDOS DEL PREPROCESADOR

Los comandos del preprocesador se utilizan para permitir realizar diversas tareas durante el proceso de la compilación entre ellas se puede establecer la configuración del programa para diversos tipos de soporte de hardware, sistemas operativos, etc. La generación de tareas mediante el uso de etiquetas (macros), la utilización de diversas constantes determinadas por el programador, la inclusión de archivos externos al archivo del programa a compilar.

Los comandos del preprocesador son los siguientes:

#include	#ifndef	#else	#line
#define	#if	#endif	#pragma
#ifdef	#elif	#error	#undef

Debemos tener en cuenta que estos comandos actúan solamente durante el proceso de la compilación y no durante el tiempo de ejecución, y por lo tanto no es posible modificar los valores de éstos mientras se ejecuta el programa.

HEADERS

Un header es un archivo de texto que se incluye en el programa a compilar, en éste se guardan los prototipos de las funciones, tipos de variables, macros y constantes que se encuentran en un archivo objeto, si es una biblioteca estándar o fuente en caso de que sea una biblioteca desarrollada por el programador.

Estos headers deben ser incluidos dentro del archivo a compilar mediante la directiva **#include** y a continuación el nombre del header a vincular, éste debe tener una extensión ".h" y debe estar entre corchetes angulares (< >) o entre comillas ("").

Su sintaxis es:

#include < nombre del header >

o

#include " nombre del header "

La diferencia entre encerrar el nombre del header entre corchetes angulares o entre comillas radica en el procedimiento de búsqueda que se utilizará para ubicarlo en el disco. Si el header está encerrado entre corchetes angulares el compilador lo buscará en los directorios determinados por la implementación del compilador, esto se utiliza para incluir los archivos de biblioteca estándar del compilador.

En el caso de que esté encerrado entre comillas procederá a buscarlo en el directorio actual, si no lo encuentra en el path del sistema operativo. Puede suceder que en este caso se le indique la unidad de disco y la ruta a seguir en cuyo caso comenzará por ella.

Ejemplo

```
.  
#include <stdio.h>
```

Aquí se indica que busque el header '*stdio.h*' en los directorios que tiene determinado el compilador a tal efecto.

```
#include "arch1.h"
```

En este caso se le indica que lo busque en el directorio que se está utilizando.

```
#include "C:\headers\arch2.h"
```

Como podemos observar le pedimos que busque al archivo en la ruta indicada en la directiva.

CONSTANTES SIMBÓLICAS

Una constante simbólica es una etiqueta que define el programador que es reemplazada por un valor constante dentro del programa.

Esto se utiliza para permitir poder realizar cambios de los valores de una constante en una única posición del programa y en el caso de que se deba modificar este valor se puede realizar antes de compilarlo y en un único lugar del mismo. De otra forma se encontraría con la desventaja de que habría que buscarla dentro del programa pudiendo dar lugar a errores involuntarios. También hay que considerar que esta forma de definir constantes permite mejorar la claridad del programa fuente.

Las constantes simbólicas se declaran mediante el uso de la directiva del preprocesador `#define`.

La sintaxis de esta es:

#define	identificador	cadena_de_sustitución
----------------	----------------------	------------------------------

La directiva se pone de manifiesto en el proceso de compilación. En este proceso se realiza una revisión del código fuente que permite al compilador producir todos los reemplazos de las constantes simbólicas antes de producir la real compilación del programa, en este paso no se verifica la sintaxis de los reemplazos de los identificadores por la cadena a sustituir, por ello es muy importante tener mucho cuidado en la utilización de una directiva que utilice la directiva `#define`. El proceso que realiza el preprocesador es el siguiente: busca dentro del archivo fuente los identificadores especificados en la directiva del preprocesador y procede a reemplazarlo con la `cadena_de_sustitución`.

Ejemplo

```
#include <stdio.h>
#define MAX 10
int main(void)
{
    int i,vec[MAX];
    for(i=0;i<MAX;i++)
        scanf("%d",&vec[i]);
    printf("\n");
    for(i=0;i<MAX;i++)
        printf("%4d\t",vec[i]);
    return 0;
}
```

Podemos observar que se definió una constante simbólica cuyo identificador se denomina MAX y la cadena de sustitución es 10. Durante la compilación Se efectuarán los siguientes pasos:

```
#include <stdio.h>
#define MAX 10
int main(void)
{
    int i,vec['10'];
    for(i=0;i<'10';i++)
        scanf("%d",&vec[i]);
    printf("\n");
    for(i=0;i<'10';i++)
        printf("%4d\t",vec[i]);
    return 0;
}
```

Primero reemplaza al identificador MAX por la cadena de sustitución 10, luego analiza el código considerando lo que realmente quedo escrito.

```
#include <stdio.h>
#define MAX 10
int main(void)
{
    int i,vec[10]
    for(i=0;i<10;i++)
        scanf("%d",&vec[i]);
    printf("\n");
    for(i=0;i<10;i++);
        printf("%4d\t",vec[i]);
    return 0;
}
```

De esta manera el identificador es reemplazado por un número, en este caso vale 10.

También podemos utilizar una constante simbólica que sea un número real.

```
#define      e      2.718281828459

#define      PI      3.14159265359
```

En ambos casos reemplazará al identificador por la cadena completa utilizando todos los decimales que hayan sido definidos.

En el caso de que se desee utilizar una menor precisión en la declaración de la constante solo hay que utilizar menor cantidad de dígitos decimales al definirla.

```
#define      e      2.71

#define      PI      3.1416
```

No solo se puede utilizar una constante simbólica para definir un valor numérico sino también una leyenda que se desee tener para indicar alguna condición de error o algún tipo de mensaje que se desee incluir dentro del código para poder hacerlo más legible al archivo fuente.

```
#define      PT      "Presione una tecla para continuar"
```

En este caso se debe tener en cuenta que el mensaje que se utiliza debe estar encerrado entre comillas para indicar que se trata de un string. Además, si se utiliza una barra invertida (\), o un símbolo de porcentual (%) dentro del mensaje se deberá escribir dos veces. Si se desea incluir comillas (") dentro del mensaje, éstas deberán estar precedidas por la barra invertida (\) para que no se confunda con la comilla de cierre del mensaje.

```
#define      PATH      "Debe indicar el path correspondiente al archivo\
ej.:\"C:\\lib\\\""
```

MACROS

Una macro es un conjunto de instrucciones de código que se indican mediante un identificador.

Puede reemplazar partes de código que se escribirán varias veces dentro del archivo fuente o también puede actuar como si fuese una función recibiendo parámetros o no.

La principal ventaja de una macro reside en que presenta una mayor velocidad de ejecución en comparación con una función que realice la misma tarea. Esto se debe a que las macros son sustituidas por su equivalente cuando se produce la compilación del programa, en cambio, una función sólo se escribe una vez y cuando hay que utilizarla se debe realizar un salto a la posición de memoria en donde está ubicada, debiendo guardar la posición a donde debe regresar y el contenido de las variables que existen en el programa.

La principal desventaja de este método consiste en que una macro es reemplazada tantas veces como es invocada y por lo tanto genera grandes cantidades de código haciendo que el programa ejecutable resulte más extenso que si se usaran funciones.

Para generar una macro utilizamos la sentencia `#define`. Anteriormente la habíamos utilizado para definir constantes.

Esta directiva tiene la siguiente sintaxis:

```
#define      identificador      codigo_substituto
```

En el identificador se escribe el nombre que vamos a utilizar en el programa fuente para invocar a la macro.

El código_substituto representa las líneas de código con las cuales serán reemplazados los identificadores.

Ejemplo

```
#define      NL           putchar('\n')

#define      TAB          putchar('\t')

#define      CARTEL       printf("Elija una opción\n")
```

En el dos primeros ejemplos se reemplazan los identificadores NL y TAB por las instrucciones putchar('\n') y putchar('\t') respectivamente. En el último se reemplaza al identificador cartel por la instrucción printf("Elija una opción\n").

```
#include <stdio.h>
#define nl putchar('\n')
#define tab putchar('\t')
#define cart printf("desea comenzar de nuevo")
int main(void)
{
    char i;
    for(i=65;i<90;i+=3)
    {
        cart;
        tab;
        printf("%c",i);
        tab;
        printf("%c",i+1);
        tab;
        printf("%c",i+2);
        nl;
    }
    return 0;
}
```

En este programa la salida sería de la siguiente forma:

desea comenzar de nuevo	A	B	C
desea comenzar de nuevo	D	E	F
desea comenzar de nuevo	G	H	I
desea comenzar de nuevo	J	K	L
desea comenzar de nuevo	M	N	O
desea comenzar de nuevo	P	Q	R
desea comenzar de nuevo	S	T	U
desea comenzar de nuevo	V	W	X
desea comenzar de nuevo	Y	Z	[

En una macro se puede utilizar también otra macro siempre y cuando haya sido definida con anterioridad a su uso.

```
#include <stdio.h>
#define MAX 100
#define NL putchar('\n')
#define TAB putchar('\t')
#define FORMATO1 NL,NL,TAB
#define FORMATO2 NL,TAB,TAB
#define COMENZAR FORMATO1,printf("Desea comenzar? "),NL,\
printf("1-SI 2-No")

#define UNO 1
#define DOS UNO+UNO
#define TRES UNO+DOS

int main(void)
{
    char l;
    NL;
    NL;
    printf("%d",MAX);
    NL;
    NL;
    printf("%c",MAX);
    COMENZAR;
    FORMATO2;
    l=getche();
    NL;
    printf("\n%d\t%d\t%d",UNO,DOS,TRES);
    return 0;
}
```

En este programa se puede observar que se han definido varias macros que realizan diferentes tareas.

Dos de ellas son constantes simbólicas una llamada MAX y la otra denominada UNO. También se definieron dos macros NL y TAB que se encargan de imprimir un carácter de nueva línea y el de tabulación respectivamente. Veamos la expansión de estas:

```
#define NL putchar('\n')
```

```
putchar('\n')
```

```
#define TAB putchar('\t')
```

```
putchar('\t')
```

Tanto las macros FORMATO1 Y FORMATO2 tienen en su definición las macros NL y TAB en combinaciones diferentes, separadas mediante el operador coma, estas macros se expanden ejecutando cada una de las sentencias, una a continuación de la otra. Cabe destacar que la macro denominada FORMATO1 se utiliza dentro de otra macro llamada COMENZAR. Su expansión será de la siguiente manera:

```
#define FORMATO1 NL,NL,TAB
```

```
putchar('\n'),putchar('\n'),putchar('\t')
```

```
#define      FORMATO2      NL, TAB, TAB
```

```
putchar('\n'), putchar('\t'), putchar('\t')
```

En la macro COMENZAR se invoca a la macro FORMATO1 una llamada a la función printf, un NL y otra llamada a la función printf, todas separadas por comas. Esta macro se expandirá de la siguiente forma:

```
#define      COMENZAR      FORMATO1, printf("Desea comenzar?"), NL, \
                           printf("1-SI  2-No")
```

```
putchar('\n'), putchar('\n'), putchar('\t'), printf("Desea comenzar?"), \
putchar('\n'), printf("1-SI  2-No");
```

Hay que indicar dos puntos importantes, uno de ellos es el uso operador coma y el otro el uso de la barra invertida.

El operador coma trabaja haciendo que la instrucción que se indica a continuación de éste corresponda a la misma definición que el que esta delante de él. Es decir que hace que se evalúen las expresiones en el orden de izquierda a derecha permitiendo que se realice una a continuación de la otra.

En el caso de que no sea suficiente el espacio en la línea para escribir las sentencias y se debe continuar en la línea siguiente, se debe colocar el carácter de barra invertida (\) para separar las dos líneas dentro de una misma sentencia. En este caso este carácter actúa como carácter de continuación, cabe destacar que este es una secuencia de escape que espera un comando determinado para realizar una acción, como el carácter que le sigue es un carácter de nueva línea y retorno de carro, lo que realiza es unir ambas líneas en una sola en la compilación.

Existen también dos macro llamadas DOS y TRES, la primera es la suma de la constante simbólica UNO dos veces, esta se expande como el número 2, la misma es 1+1. La denominada TRES es la suma de las constantes UNO y DOS, dando como resultado el número 3. Se expanden de la siguiente manera:

```
#define      DOS      UNO+UNO
```

```
1+1
```

```
#define      TRES      UNO+DOS
```

```
1+1+1
```

el programa queda luego de las expansiones de las macros de la siguiente forma, para luego proceder a la compilación y el linkeo correspondiente.

```
int main(void)
{
    char l;
    putchar('\n');
    putchar('\n');
    printf("%d", 100);
    putchar('\n');
    putchar('\n');
```

```
printf("%c",100);
putchar('\n'),putchar('\n'),putchar('\t'),printf("Desea comenzar? "),
putchar('\n'),printf("1-SI 2-No");
putchar('\n'),putchar('\t'),putchar('\t');
l=getche();
putchar('\n');
printf("\n%d\t%d\t%d",1,1+1,1+1+1);
return 0;
}
```

PASAJE DE PARÁMETROS A UNA MACRO

Una posibilidad muy importante que poseen la macro es la de poder recibir parámetros cuando es invocada. Permitiendo su utilización de la misma forma que si fuese una función. Para ello hay que ponerle entre paréntesis uno o varios parámetros, los que en el código_sustituto trabajarán como elementos que tomarán los diferentes valores dentro de las operaciones que se deseen realizar.

En el siguiente ejemplo podemos ver una aplicación de ellas:

```
#define LEER_DATO(ans) scanf("%d",&ans)
```

Esta macro se podría llamar de la siguiente forma:

```
LEER_DATO(x)
```

Expandiéndose:

```
scanf("%d",&x)
```

Veamos a continuación otro ejemplo:

```
#define CUADRADO(X) X * X
```

En este ejemplo si se lo utiliza con un valor numérico funcionaría correctamente. Es decir, si lo invocamos de esta forma:

```
Y=CUADRADO(5);
```

Obtendríamos la siguiente respuesta en la variable 'y':

```
Y=25
```

Pero si lo utilizamos de la siguiente forma:

```
V=3;
Y=CUADRADO(V+1);
```

La respuesta sería:

```
Y=7
```

Debido a que el compilador entiende:

$Y = V + 1 * V + 1$

Para solucionar esto deberíamos definir a la macro parametrizando cada variable

```
#define CUADRADO (X) (X) * (X)
```

Si la invocamos como anteriormente, tendríamos una expansión correcta de lo pedido puesto que el compilador la entendería de esta forma:

```
V=3;
Y=CUADRADO (V+1) ;
```

$Y = (V+1) * (V+1)$

Dando como resultado 16 que es el valor esperado.

Es conveniente encerrar este tipo de macro entre paréntesis debido a que si uno desea castear la expresión lo haría correctamente.

```
#define CUADRADO (X) ((X) * (X))
```

```
H=(int) CUADRADO (J) ;
```

Si casteamos la macro se expandiría:

$H = (int)((J) * (J));$

Tampoco podemos dejar espacios en blanco en el identificador, puesto que daría errores de interpretación en la compilación.

Si se define de esta forma produciría un error.

```
#define LEER_DATO (ans) scanf ("%d", &ans)
```

```
LEER_DATO (a) ;
```

Se expandirá como :

$(ans) \text{ scanf} ("%d", \&ans) (a);$

Produciendo errores en la compilación.

También si se utilizan los operadores de incremento o decremento podrían producir errores debido a que los compiladores pueden interpretar una macro de diferentes formas.

```
X=5;
Y=CUADRADO (X++) ;
```

Podría expandirse en una de estas posibles formas:

$((5) * (6))$
 $Y = 30$

0

```
((5) * (5))
Y=25
```

Dada la imposibilidad de predecir cuál de las dos formas es la que obtendremos, debemos evitar el uso de estos operadores en nuestras macros.

Cuando se utiliza una función en un programa, además de podersele pasar valores, también es posible declarar variables dentro de ellas; en una macro también se lo puede hacer, solo hay que tener en cuenta encerrarla entre dos llaves, esto define un bloque y dentro de uno se pueden declarar variables, las que son locales al bloque.

Las macros presentan la ventaja de independizarse del tipo de variables que se le pasan, es decir si uno pensó una macro que reciba variables de tipo int y en algún otro caso se le desean enviar de tipo float lo haría sin inconveniente alguno, debido a que una macro se expande en el lugar del programa en donde se la invocó y no debe enviar valores de variables al stack para luego recuperarlos.

Ejemplo

```
#include <stdio.h>
#include <conio.h>
#define INGRE(val) scanf("%d",&val)
#define MAX 10
#define LAZO(b) {int a;for(a=0;a<MAX;a++) INGRE(b[a]);}
#define SORT(v) {int a,b,aux;for(a=0;a<MAX-1;a++)\
for(b=a+1;b<MAX;b++) if(v[a]<v[b])\
{aux=v[b];v[b]=v[a];v[a]=aux;}}
#define SAL(b) {int a;for(a=0;a<MAX;a++)printf("\t%d",b[a]);}
int main(void)
{
    int vec[MAX];
    clrscr();
    LAZO(vec);
    SORT(vec);
    SAL(vec);
    return 0;
}
```

En el ejemplo anterior podemos observar que se han definido varias macros una llamada INGRE que permite el ingreso de un valor entero por teclado, una constante simbólica MAX que tomará el valor 10, y las tres últimas LAZO, SORT y SAL que en realidad actuaran como si fuesen funciones. Es importante ver que todas ellas están encerradas entre llaves, para permitir que sean tratadas al expandirse como bloques. Además, tiene la particularidad de que es posible definir variables locales a cada uno de los bloques.

En la macro SORT está escrito un if, hay que tener cuidado con esta sentencia en particular porque entre la condición y el bloque de sentencias afirmativo hay que dejar por lo menos un carácter de separación entre ellos, si no, no reconocerá la sintaxis correcta de la instrucción y producirá un error el cual es muy difícil de encontrar. Los caracteres de separación habituales son el retorno de carro - nueva línea, el espacio o la tabulación.

La macro LAZO permite el ingreso de elementos en un vector. La llamada SORT realiza el ordenamiento de los elementos de dicho vector, y la denominada SAL realiza la impresión en la pantalla de tales valores.

El programa con las macros expandidas quedaría de la siguiente forma:

```
int main(void)
{
    int vec[10];
    clrscr();
    {int a;for(a=0;a<10;a++)scanf("%d",&vec[a]);};
    {int a,b,aux;for(a=0;a<10-1;a++) for(b=a+1;b<10;b++) if(vec[a]<vec[b])\
    {aux=vec[b];vec[b]=vec[a];vec[a]=aux;}};
    {int a;for(a=0;a<10;a++)printf("\t%d",vec[a]);};
    return 0;
}
```

MACROS PREDEFINIDAS POR EL COMPILADOR

El estándar ANSI define cinco macros que provee todo compilador que cumpla con esas normas, que son:

__LINE__ **__FILE__** **__DATE__** **__TIME__** **__STDC__**

Como se puede apreciar comienzan y terminan con dos caracteres de subrayado y además no reciben parámetros.

__LINE__ : Esta devuelve el valor del número de línea que posee la línea actual que se está ejecutando, tiene un formato entero.

__FILE__ : Retorna el nombre del archivo fuente que se ha compilado, tiene el formato de un string.

__DATE__ : Entrega la fecha de la compilación del programa que se ejecuta, tiene el formato de un string de la forma MM DD YY.

__TIME__ : Devuelve la hora en que se compiló el programa, tiene el formato de un string y toma el formato hh:mm:ss.

__STDC__ : Esta macro retorna un valor 1 si el compilador es compatible con el estándar ANSI, caso contrario devuelve un valor 0, tiene el formato de un entero.

COMPILACIÓN CONDICIONAL

En muchas ocasiones encontramos la necesidad de tener que generar un programa que permita su utilización en diferentes entornos, ya sea diferentes tipos de hardware o diversos sistemas operativos. Para lograr esto se debería escribir varias veces el mismo archivo fuente, con la consiguiente pérdida de tiempo y esfuerzo. En otras ocasiones se precisa disponer de varias opciones, dentro de un mismo esquema general que se ejecuten según las necesidades del usuario, en este caso se puede optar por escribir diversos

archivos, uno para cada situación en particular, con los mismos inconvenientes que mencionamos anteriormente.

Para solucionar ambas situaciones existe lo que se denomina compilación condicional, ella permite al programador satisfacer ambas necesidades con solo indicarle al compilador mediante etiquetas que parte compilar y cual no, de acuerdo a esto se puede escribir un único archivo fuente que sirva tanto para diversas plataformas, tanto de software como de hardware, o quizás también permitirle al programador la posibilidad de poder satisfacer distintas situaciones con un único archivo fuente.

Para poder realizar esto el lenguaje "C" cuenta con algunas directivas del preprocesador, las que son:

#if	#ifndef	#else	#define
#ifdef	#elif	#endif	#undef

Creación de etiquetas

Aunque hemos visto que la directiva **#define** sirve tanto para crear constantes simbólicas como macros también tiene la particularidad de permitir la creación de etiquetas que sirven para generar una indicación para que se pueda compilar de una forma u otra.

Su sintaxis será:

#define etiqueta

Esta directiva se utiliza para permitir verificar si existe o no dicha etiqueta y poder generar el archivo objeto con las sentencias adecuadas según su existencia o no.

Directivas condicionales

Las directivas **#if**, **#ifdef** y **#ifndef** son condicionales, al igual que la sentencia **if** que se utiliza en la programación. Estas evalúan la condición que tienen escrita y permitirán la compilación del código que le sigue en caso de que esta sea verdadera y en caso contrario no será compilado. Las directivas anteriormente nombradas deben tener un final para su cuerpo, para ello existe la directiva **#endif** que sirve para dar fin a la directiva condicional. Hay que tener en cuenta que a cada **#if** le corresponde un **#endif**.

Las diferencias que existen entre ellas son mínimas, pero conviene aclarar que tanto **#ifdef** como la directiva **#ifndef** solamente pueden evaluar la existencia de una etiqueta dada por una directiva **#define**. **#ifdef** tomará un valor verdadero si y solo si existe la etiqueta por la cual se está interrogando, en cambio **#ifndef** será verdadera siempre que la etiqueta no este definida con anterioridad.

La directiva **#if** permite no solo la verificación de la existencia o no de una etiqueta sino también evaluar el valor de la misma, y permite la utilización de operadores relacionales como ser los operadores de igualdad (**==**), de desigualdad (**!=**), mayor que (**>**), menor que (**<**), sino también los operadores lógicos **and** (**&&**), **or** (**||**), y de negación (**!**). Cuando se quiere determinar si existe una etiqueta se debe utilizar con la directiva **#if** la palabra reservada **defined** y a continuación la etiqueta por la que se quiere interrogar.

Si cuando se utiliza una directiva condicional se quiere ejecutar dos opciones distintas según el resultado de la evaluación que se realizó es necesario poder tener dos cuerpos diferentes uno para la respuesta verdadera y el otro para la falsa, para ello existe la directiva `#else`. La directiva `#elif` permite el anidamiento de expresiones condicionales, el equivalente a ella sería la sentencia `else if` que se utiliza en código "C". Estas directivas pueden estar ubicadas en cualquier parte del archivo fuente.

La sintaxis para estas directivas es:

```
#if    condición
bloque de sentencias a compilar (verdadero)
#endif

#if    condición
bloque de sentencias a compilar (verdadero)
#else
bloque de sentencias a compilar (falso)
#endif

#if    condición
bloque de sentencias a compilar (verdadero)
#elif  condición 2
bloque de sentencias a compilar (verdadero 2)
#endif
#endif

#if    condición
bloque de sentencias a compilar (verdadero)
#elif  condicion2
bloque de sentencias a compilar (verdadero 2)
#else
bloque de sentencias a compilar (falso 2)
#endif

#if    defined etiqueta
bloque de sentencias a compilar (verdadero)
#endif

#ifdef    etiqueta
bloque de sentencias a compilar (verdadero)
#endif

#ifndef    etiqueta
bloque de sentencias a compilar (verdadero)
#endif
```

Hay que tener en cuenta que se puede utilizar cualquier combinación de directivas según sea necesario.

Ejemplo

```
#include<stdio.h>
#define MAX 15
int main(void)
{
    #ifdef MAX
        int a;
        printf("\n");
        for(a=1;a<MAX;a+=2)
            printf("%d\t",a);

    #else
        #define MAX 20
        printf(MAX);
    #endif
    #undef MAX
    #ifdef MAX
        int a;
        printf("\n");
        for(a=1;a<MAX;a+=2)
            printf("%d\t",a);

    #else
        #define MAX 20
        printf("\n\n%d",MAX);
    #endif
    return 0;
}
```

En este ejemplo se puede observar el uso de las directivas `#ifdef`, `#else`, `#endif`, `#define` y `#undef`.

Aquí definimos una constante simbólica llamada `MAX` con un valor 15. Dentro de la función `main()` se utiliza el conjunto `#ifdef - #else - #endif`, en dos ocasiones, en ambas se realiza la misma secuencia de líneas de programa. Dentro de la resolución verdadera de la directiva `#ifdef` se declara una variable de tipo `int`, luego se imprime un salto de línea, y por último se realiza un lazo `for` imprimiendo un conjunto de números enteros. En la resolución falsa de esta directiva condicional se define la `MAX` con el valor 20 y luego se imprime este valor.

Entre las directivas condicionales se indefinire a la etiqueta `MAX`. Esta directiva se utiliza para poder visualizar cómo funciona la directiva `#ifdef`.

En la primera se compilará sólo la resolución verdadera mientras que en la segunda sólo la resolución falsa. Dando como resultado la siguiente salida:

```
1    3    5    7    9    11   13
```

```
20
```

```
#include    <stdio.h>
int main(void)
{
    #ifndef MAX
        #define MAX 19
        int a;
        printf("\n");
        for(a=1;a<MAX;a+=2)
            printf("%d\t",a);
```

```

#else
#define MAX 20
printf(MAX);
#endif
#ifndef MAX
    int a;
    printf("\n");
    for(a=1;a<MAX;a+=2)
        printf("%d\t",a);
#else
#define MAX 20
printf("\n\n%d",MAX);
#endif
return 0;
}

```

Podemos observar cómo actúa la directiva `#ifndef`, al igual que en el ejemplo en donde se observaba la directiva `#ifdef`, este realiza la misma secuencia. Pero como en un principio no se definió a la etiqueta `MAX` actúa la resolución verdadera de la directiva. Dando como resultado:

```
1    3    5    7    9    11   13   15   17
```

```
20
```

```

#include <stdio.h>
#define TIPO 1
int main(void)
{
    #if (TIPO==1)
    printf("\nEl valor de TIPO es %d ",TIPO);
    #else
    #define TIPO 4
    printf("\nEl valor de TIPO es %d ",TIPO);
    #endif
    printf("\n\n");
    #undef TIPO
    #define TIPO 2
    #if (TIPO>0)
    printf("\nEl valor de TIPO es %d ",TIPO);
    #else
    #define TIPO 4
    printf("\nEl valor de TIPO es %d ",TIPO);
    #endif
    printf("\n\n");
    #undef TIPO
    #define TIPO 2
    #if (TIPO==0)&&(TIPO!=2)
    printf("\nEl valor de TIPO es %d ",TIPO);
    #else
    #define TIPO 4
    printf("\nEl valor de TIPO es %d ",TIPO);
    #endif
    return 0;
}

```

Podemos encontrar en este ejemplo la utilización de la directiva `#if`, se puede observar que mediante el uso de la constante simbólica `TIPO` toma diversos valores, mediante el uso de las directivas `#undef` y `#define`. Ello permite ver cómo actúa la directiva `#if` con diversas condiciones, las dos primeras son condiciones simples mientras que la última se usa una condición compuesta mediante el operador booleano `and (&&)`.

La salida de este ejemplo es:

El valor de TIPO es 1

El valor de TIPO es 2

El valor de TIPO es 4

Directiva `#undef`

La directiva `#undef` se utiliza para permitir anular una constante simbólica, macro, o etiqueta desde el lugar en donde se ha ubicado esta directiva.

Su sintaxis es:

`#undef` etiqueta

o

`#undef` identificador

Ejemplo

```
#include<stdio.h>
#define MAX 100
int main(void)
{
    int i;
    printf("\n");
    for(i=0;i<MAX;i++)
        printf("%d\t",i);
    #undef MAX
    #define MAX 20
    printf("\n");
    for(i=0;i<MAX;i++)
        printf("%d\t",i);
    return 0;
}
```

Podemos observar que se definió una etiqueta llamada `MAX` como constante simbólica con un valor 100.

Este valor existe hasta que se escribe la directiva `" #undef MAX "` que elimina la etiqueta `"MAX"`. A partir de esta línea no existe dicha etiqueta mientras no haya sido

redefinida, lo que ocurre con la directiva "#define MAX 20" y a partir de esta toma un nuevo valor.

DIRECTIVAS PARA DETECCION DE ERRORES

Estas directivas sirven para poder detectar errores durante el proceso de compilación. Ellas actúan cuando se compila sin la utilización del entorno integrado de desarrollo. En caso contrario ellas no intervienen durante el proceso de desarrollo.

Directiva #line

Sirve para cambiar el número de la línea del programa fuente a compilar.

Su sintaxis es:

#line numero_de_linea

Donde número_de_línea es un número entero que será el nuevo número de la línea que corresponde al programa fuente que se compilará.

Esta directiva tiene el efecto deseado solo cuando se produce la compilación desde la línea de comandos del sistema operativo.

Ejemplo

```
#include <stdio.h>
#line 100
int main(void)
{
    int i;
    for(i=0;i<10;i++)
        printf("%d",i);
    printf("\n%d\n",__LINE__);
    #line 200
    for(i=0;i<10;i++)
        printf("%d",i);
    printf("\n%d\n",__LINE__);
    #line 300
    printf("\n%d\n",__LINE__);
    return 0;
}
```

En la segunda línea se declara la directiva #line con el valor 100 a partir de esta se comenzarán a numerar las líneas a partir de dicho número, imprimiéndose en la sentencia " printf("\n%d\n",__LINE__); " el número de la línea correspondiente a ella que en este caso será 105.

En la línea siguiente se vuelven a renumerar a las líneas, pero a partir del número 200 y luego se vuelve a renumerar, pero con el número 300.

A continuación, se detalla el listado del ejemplo, pero con el agregado de los números de línea.

```
1.    #include      <stdio.h>
2.    #line         100
100.  int main(void)
101.  {
102.      int i;
103.      for(i=0;i<10;i++)
104.          printf("%d",i);
105.      printf("\n%d\n",__LINE__);
106.      #line      200
200.      for(i=0;i<10;i++)
201.          printf("%d",i);
202.      printf("\n%d\n",__LINE__);
203.      #line      300
300.      printf("\n%d\n",__LINE__);
301.      return 0;
302.  }
```

Su salida es:

```
0123456789
105
0123456789
202

300
```

Directiva #error

Sirve para generar mensajes de error cuando se compila un programa. Cabe destacar que cuando se encuentra esta línea se detiene la compilación del programa.

Su sintaxis es:

#error mensaje_de_error

Donde mensaje_de_error es el mensaje generado por el usuario para denominar el error producido.

Ejemplo

```
#include<stdio.h>
#define MAX 100
#line 100
int main(void)
{
    int i;
    for(i=0;i<10;i++)
        printf("%d",i);
    printf("\n%d\n",__LINE__);
    #error mensaje de error
    for(i=0;i<10;i++)
```

```
    printf("%d", i);  
    printf("\n%d\n", __LINE__);  
    #line 300  
    printf("\n%d\n", __LINE__);  
    return 0;  
}
```

Este ejemplo detendrá su compilación cuando se encuentra la directiva `#error` indicando el mensaje de error que contienen esta.

DIRECTIVA #PRAGMA

Esta directiva sirve para permitir modificar la implementación del compilador, para cada programa fuente, sin importar la implementación por omisión del compilador. Esta directiva se utiliza cuando no se utiliza para generar el programa el entorno integrado de desarrollo.

Su sintaxis es:

#pragma nombre_de_la_directiva

Donde `nombre_de_la_directiva` es el cambio pretendido a la implementación. Estos pueden ser:

#pragma	argsused
#pragma	comment
#pragma	option
#pragma	saveregs
#pragma	warn

#pragma argsused

La utilización de `argsused` se puede utilizar solamente entre las definiciones de funciones y ella permite eliminar los mensajes de advertencia, teniendo efecto solo en la función subsiguiente.

#pragma comment

Permite guardar un comentario en el archivo de salida.
Puede tomar uno de los siguientes valores:

`exestr:` El linker escribe el string en el archivo objeto, y además es ubicado en el archivo ejecutable. Sirve para permitir una rápida utilidad de búsqueda.

`lib:` Escribe un comentario en el archivo objeto. Esto permite al linker utilizarla como el directorio de búsqueda de una biblioteca. Permitiendo incluir el módulo especificado de biblioteca en este programa.

`user:` Escribe una cadena en el programa objeto, el que es ignorado por el linker.

#pragma option

Se utiliza para incluir una opción de la línea de comandos dentro del código del programa.

#pragma saveregs

Permite que no cambien los contenidos de los registros del microprocesador cuando se utiliza una función que contiene código en assembler.

#pragma warn

Permite sobre escribir los mensajes de advertencia.